02_last_class_refresher

September 18, 2026

1 Warmup: fundamentals

Recall that last week's objectives were:

- Describe fundamental aspects of a working Python script
- Implement novel functions and for loops along with appropriate unit tests
- Utilize 'external' python packages

This warm-up aims to build on those objectives and reinforce them.

1.1 1. Basics of functions

1.1.1 1(a): ins and outs

Run the following cell. It doesn't work as expected. Explain why (in a markdown cell below it) and create a new cell below that with a fixed version of the code.

[1]:
```
def factorial(n):
    """Calculates the factorial of a number."""
    fac = 1
    if n == 1:
        fac = 1
    else:
        fac = n * factorial(n-1)


# test the factorial function; this should print 120
print(factorial(5))
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[1], line 11
      7         fac = n * factorial(n-1)
     10 # test the factorial function; this should print 120
---> 11 print(factorial(5))

Cell In[1], line 7, in factorial(n)
      5     fac = 1
      6 else:
----> 7     fac = n * factorial(n-1)
```

```
Cell In[1], line 7, in factorial(n)
      5     fac = 1
      6 else:
----> 7     fac = n * factorial(n-1)

    [… skipping similar frames: factorial at line 7 (1 times)]

Cell In[1], line 7, in factorial(n)
      5     fac = 1
      6 else:
----> 7     fac = n * factorial(n-1)

TypeError: unsupported operand type(s) for *: 'int' and 'NoneType'
```

The function computes fac but never returns it, so every call returns None so the recursion fails when trying to multiple an int by None. To fix this you need to return the result of fac.

put answer to 1a here

[2]:
```python
def factorial(n):
    """Calculates the factorial of a number."""
    if n < 0:
        raise ValueError("factorial is not defined for negative numbers")
    if n <= 1:          # base case: 0! = 1! = 1
        return 1
    return n * factorial(n - 1)

print(factorial(5))
```

```
120
```

1.1.2 1(b): Writing new functions

Write a function representing the formula $z = x^2 + y^2$ and verify that it works as expected.

[3]:
```python
import math

def z(x, y):
    """Computes z = x**2 + y**2."""
    return x**2 + y**2


# check against hand computed values
cases = [
    ((0, 0), 0),
    ((1, 0), 1),
    ((0, 1), 1),
```

```
    ((3, 4), 25),
    ((-3, 4), 25),
    ((3, -4), 25),
    ((-3, -4), 25),
]
for (x, y), expected in cases:
    assert math.isclose(z(x, y), expected), (x, y, expected)
#Verify that all cases match my correct hand picked values, and print a␣
 ↪statement of validity.
print("All hand-computed values OK")
```

```
All hand-computed values OK
```

1.1.3 1(c): Scope

We haven't yet discussed the concept of *scope*, so we'll introduce it here through examining an interesting effect (and we'll discuss it more as time goes on.)

Run the cell below. We see that the value of `a` is redefined when `redefine_and_print_a()` is called, so why does the last print statement contain the original definition of `a` and not the one it was given within the function?

[4]:
```
# define a as a global variable here
a = "This is `a` with global scope."
# print a to show its value
print(f"1: {a}")

# define a function
def redefine_and_print_a():
    # define a as a local variable here
    a = "This is `a` with local scope."
    # print a to show its value
    print(f"2: {a}")

# call the function
redefine_and_print_a()

# print a one more time to show its value
print(f"3: {a}")
```

```
1: This is `a` with global scope.
2: This is `a` with local scope.
3: This is `a` with global scope.
```

I think the global and local scope is being manipulated so that there are two different variables that have 'a' defined one being for globally within the code and one being a seperate local function that is only accessible by calling redefine_and_print_a().

1.2 2. Using external packages

Make a plot of the following equation using `numpy` and `matplotlib`. Use `x` values ranging from -8 to 8, and make plots using the following values of σ: $\sigma = [0.25, 0.5, 1, 2, 4]$

$$N(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{x^2}{2\sigma^2}}$$

Your solution should (a) define a new function to represent this formula, and (b) use a `for` loop.

If you have time, customize the x- and y-axis labels, and add a legend.

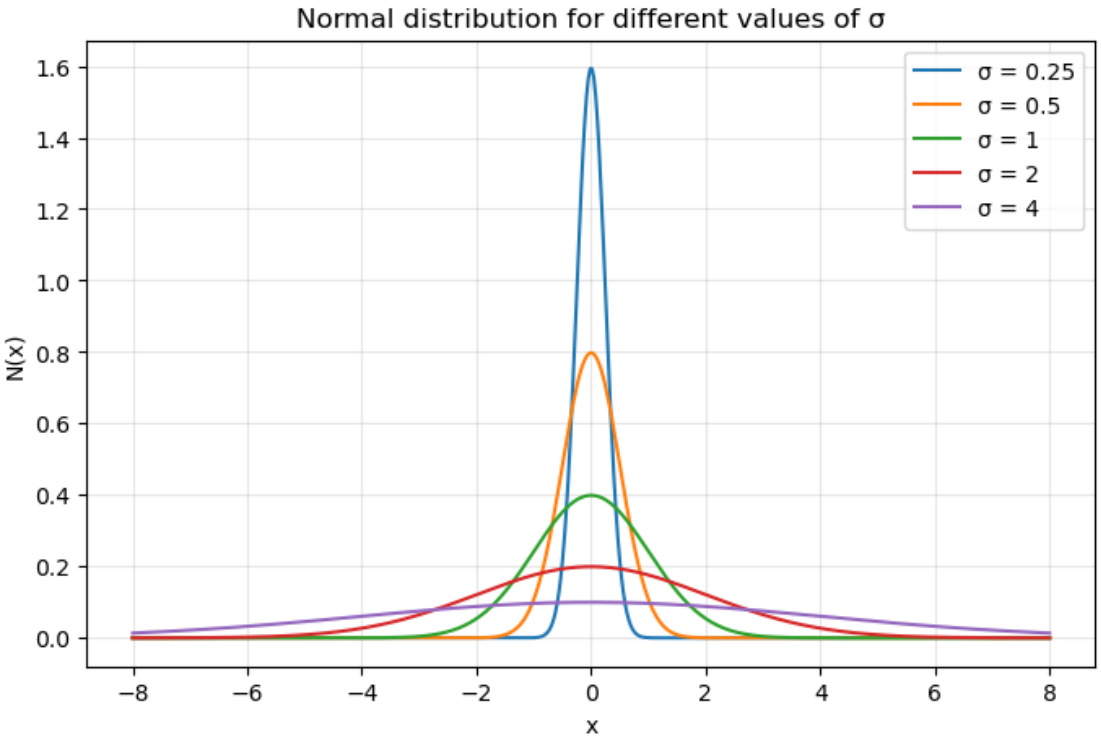
[5]:
```
import numpy as np
import matplotlib.pyplot as plt

def N(x, sigma):
    """Normal (Gaussian) distribution with mean 0 and standard deviation sigma.
    """
    return 1 / (sigma * np.sqrt(2 * np.pi)) * np.exp(-x**2 / (2 * sigma**2))

x = np.linspace(-8, 8, 500)          # x values from -8 to 8
sigmas = [0.25, 0.5, 1, 2, 4]

fig, ax = plt.subplots(figsize=(8, 5))
for sigma in sigmas:
    ax.plot(x, N(x, sigma), label=f"  = {sigma}")

ax.set_xlabel("x")
ax.set_ylabel("N(x)")
ax.set_title("Normal distribution for different values of  ")
ax.legend()
ax.grid(alpha=0.3)
plt.show()
```


Normal distribution for different values of σ
σ = 0.25
σ = 0.5
σ = 1
σ = 2
σ = 4
N(x)
x
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
−8
−6
−4
−2
0
2
4
6
8

[ ]: